

In this case, we call the `fillsquare()` function using the existing `square` variable that we defined previously. Remember we defined it to be full of `nan` entries except for five central elements that we specified. After we run the `fillsquare()` function with that `square` as its input, the `fillsquare()` function fills in many of the remaining entries. Let's print out the resulting square and see what it looks like afterward:

```
printsquare(square)
```

The result is as follows:

	[0]	[1]	[2]	[3]	[4]	[5]	[6]
[0]	22	nan	16	nan	10	nan	4
[1]	nan	23	nan	17	nan	11	nan
[2]	30	nan	24	49	18	nan	12
[3]	nan	31	7	25	43	19	nan
[4]	38	nan	32	1	26	nan	20
[5]	nan	39	nan	33	nan	27	nan
[6]	46	nan	40	nan	34	nan	28

You'll notice that the `nan`s occupy alternating entries, like a checkerboard. The reason for this is that the rules we have for moving diagonally give us access to only about half of the total entries, depending on which entry we started with. The valid moves are the same as in checkers: a piece that starts on a dark square can move diagonally to other dark squares, but its diagonal moving pattern will never allow it to move to any of the light squares. The `nan` entries we see are inaccessible if we start on the central entry. We specified $(n**2)/2-4$ for our `howfull` argument instead of zero because we know that we wouldn't be able to fill the matrix completely by calling our function only once. But if we start again on one of the central entry's neighbors, we will be able to access the rest of the `nan` entries in our "checkerboard." Let's call the `fillsquare()` function again, this time starting on a different entry and specifying our fourth argument as zero, indicating that we want to completely fill our square:

```
entry_i = math.floor(n/2) + 1
entry_j = math.floor(n/2)

square = fillsquare(square, entry_i, entry_j, 0)
```

If we print our square now, we can see that it is completely full: